ntic master
revolucionamos la comunicación

UNIVERSIDAD COMPLUTENSE DE MADRID

TAREA

DISEÑO DE INGESTAS Y LAGOS DE DATOS

Autor: Víctor Daniel Rodríguez Pérez

Índice

Introducción....3
Orígenes de datos....4
Datos para procesamiento batch....4
Datos para procesamiento Streaming....5
Diseño de la arquitectura del Lakehouse....11
1. Separación de responsabilidades y control de calidad....11
2. Trazabilidad y gobernanza de datos....11
3. Mejora del rendimiento y la eficiencia....12
4. Soporte nativo para batch y streaming....12
5. Escalabilidad y mantenimiento....12
6. Compatibilidad con arquitectura Lakehouse y herramientas modernas....13
Flujo de los datos....13
- Datos batch....13
- Datos streaming....14
Capas de la arquitectura....14
Capa landing....14
Esquema de la Arquitectura....15
Implementación del motor de ingesta....15
Nivel 1: Parámetros Globales....16
Nivel 2: data_type_information....16
Dominios disponibles:....16
Ejemplos:....17
Integración de Datos Batch....19
Integración de Datos Streaming....22
Deserialización de datos de Kafka....25
Ejemplo de configuración de jobs....25
Mejoras pendientes....26

Introducción

El presente trabajo tiene como finalidad el diseño e implementación de un sistema de ingesta de datos sobre una arquitectura *lakehouse* en la plataforma Azure Databricks. Esta práctica forma parte del módulo de Diseño de Ingestas y Lagos de Datos del Máster en Big Data y Data Engineering, y tiene como objetivo evaluar la capacidad del alumno para aplicar, de forma integrada, conocimientos técnicos relacionados con el tratamiento de datos en entornos distribuidos.

El desarrollo se divide en dos bloques principales. En primer lugar, se presenta el diseño de la arquitectura del *data lakehouse*, incluyendo la definición y justificación de las distintas capas y su representación esquemática. En segundo lugar, se implementa un motor de ingesta programado en Python, basado en Apache Spark, capaz de gestionar flujos de datos tanto en modo *batch* como *streaming*. Este motor permite configuraciones dinámicas por conjunto de datos, control de esquemas, enriquecimiento con metadatos, y el uso de tecnologías como Autoloader y Kafka.

Todas las decisiones técnicas adoptadas en esta práctica se han tomado considerando que el sistema está destinado a una empresa ficticia —FarmIA— con un alto volumen de datos y previsión de crecimiento exponencial. Por ello, se ha priorizado el diseño de una solución escalable, modular, resiliente y eficiente, tanto en el procesamiento como en el almacenamiento.

El trabajo incluye además una guía de ejecución que permite desplegar la solución como un *job* en Azure Databricks a partir de un paquete distribuible (.whl), respetando las convenciones profesionales requeridas para su uso en entornos reales o simulados.

Orígenes de datos

En el enunciado de la tarea nos definen los siguientes tipos de datos. A la hora de elegir el tipo de dato, hemos primado el tener la mayor variabilidad posible de ellos para hacer el motor de ingesta lo más robusto posible:

CONCEPTO	TIPO	PERIODICIDAD	FORMATO
Datos de ventas online desde su plataforma de comercio electrónico	batch	1h	JSON
Registros de inventario desde su sistema de gestión local	batch	1h	CSV
Sensores IoT en los campos agrícolas que monitorean temperatura, humedad y calidad del suelo	streaming	CONTINUO	JSON
Eventos de clientes en tiempo real provenientes de su aplicación móvil y redes sociales.	streaming	CONTINUO	AVRO
Datos de proveedores y logística con información sobre envíos y tiempos de entrega.	batch	1h	CSV
Información meteorológica externa para predecir cosechas y optimizar la distribución.	batch	1h	JSON

Para conseguir generar los tipos de datos especificados anteriormente, lo hacemos de la siguiente manera:

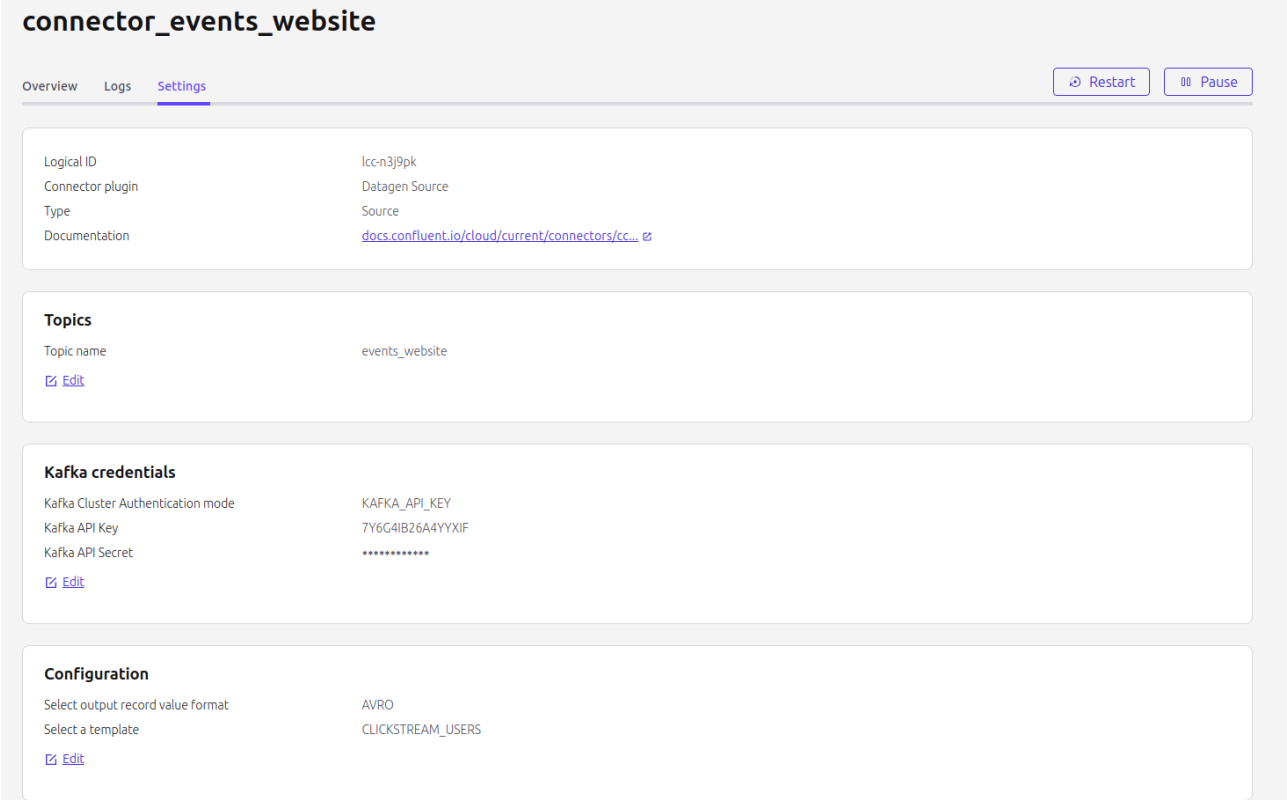
Datos para procesamiento batch

A la hora de generar los archivos para la posterior ingesta, hemos usado el notebook adjunto *generador_datos_batch.ipynb.* Al ejecutarlo, se generan 10 archivos que se escriben en el contenedor landing de nuestra Cuenta de Almacenamiento en Azure, de donde serán leídos posteriormente.

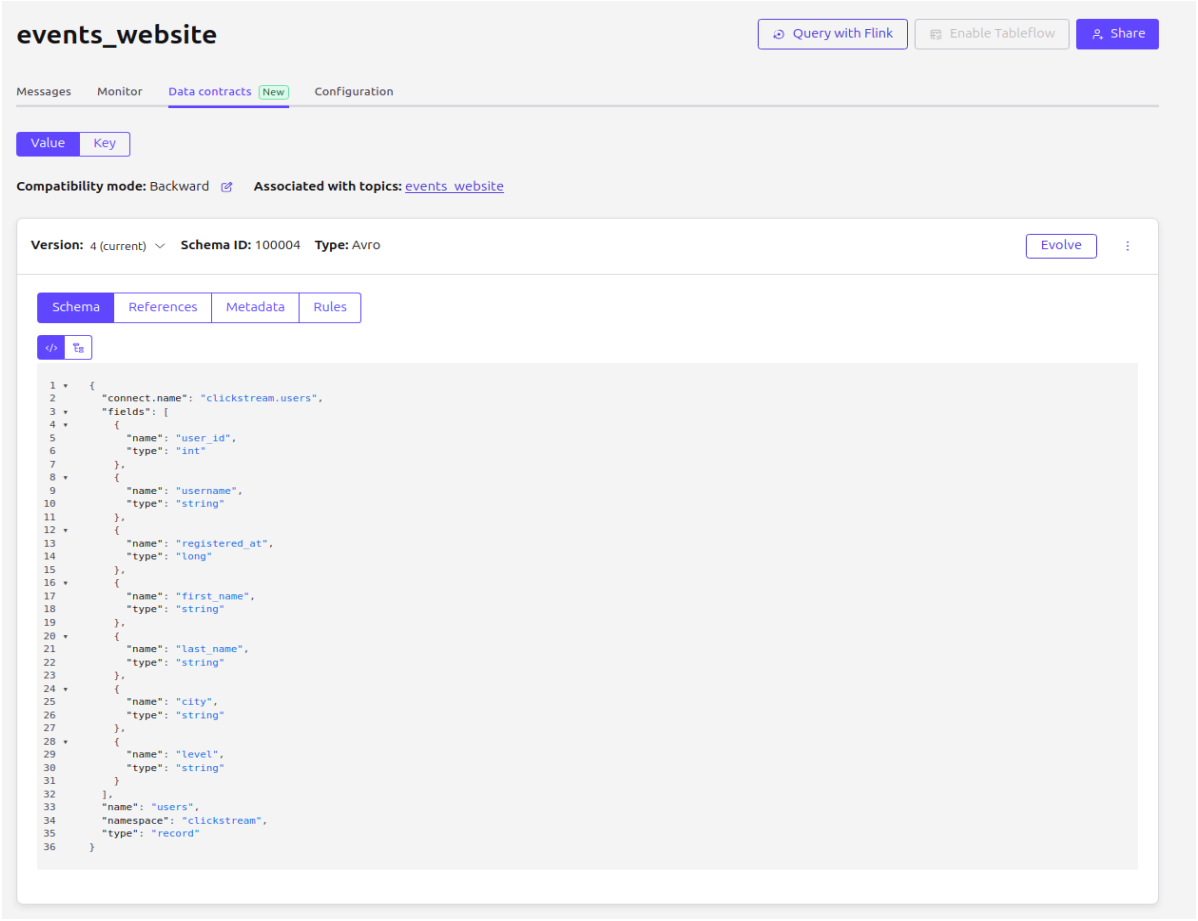
Datos para procesamiento Streaming

Para generar el contenido que se va a leer desde Kafka, hemos optado por usar Confluent Kafka, haciendo uso de sus conectores, en conjunto con el plugin para crear sample data:

Para los mensajes de eventos en tiempo real provenientes de provenientes de su aplicación móvil y redes sociales, utilizamos uno de los *sample quick schema* que nos proporciona Confluent: ClickStream Users. Este conector vuelca los mensajes al topic *events_website en formato avro.*



Al utilizar avro, generamos un Data Contract en Schema Registry, con compatibilidad Backward. Ello nos permite en nuestro código, venir a Schema Registry a buscar la estructura del mensaje para el parseo.

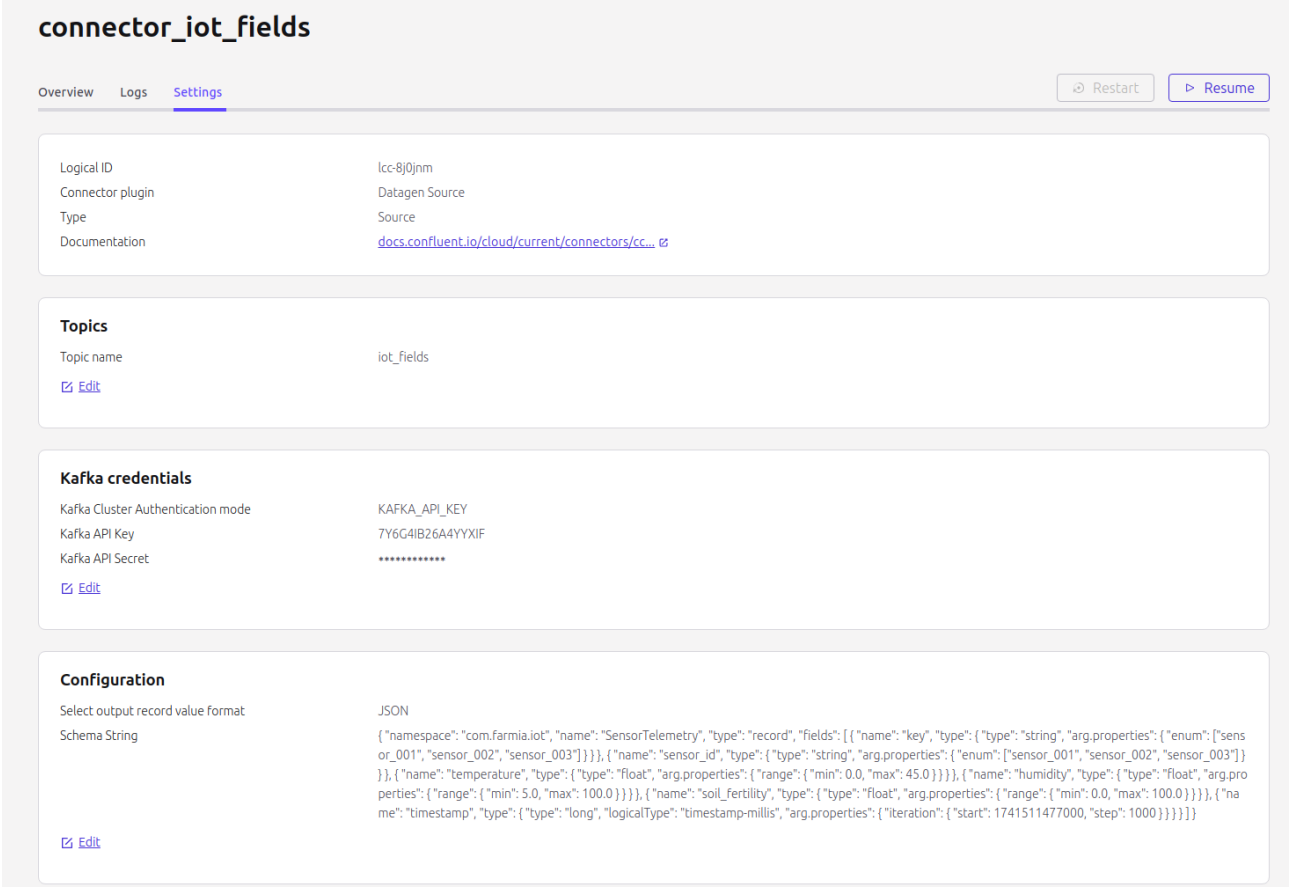


Para los mensajes de de sensores Iot de los campos agrícolas, utilizamos también un conector con el plugin sample data, pero esta vez en formato json y en vez de utilizar uno de los *sample quick schema* como con el tipo de dato anterior, le pasamos nuestro esquema personalizado:

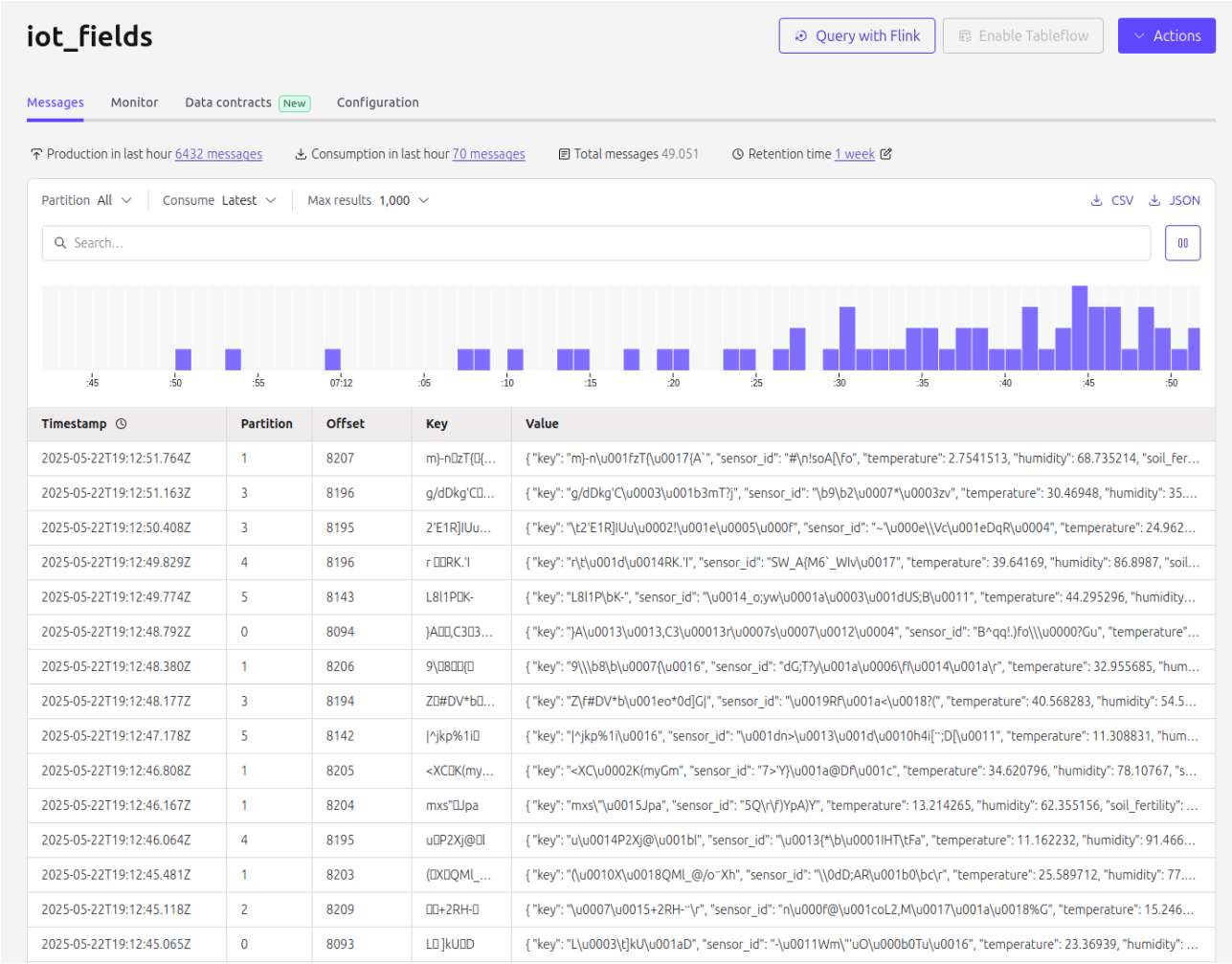
```
{
  "namespace": "com.farmia.iot",
  "name": "SensorTelemetry",
  "type": "record",
  "fields": [
    {
      "name": "key",
      "type": {
        "type": "string",
        "arg.properties": {
```

```
      "enum": ["sensor_001", "sensor_002", "sensor_003"]
    }
   }
  },
  {
   "name": "sensor_id",
   "type": {
    "type": "string",
    "arg.properties": {
     "enum": ["sensor_001", "sensor_002", "sensor_003"]
    }
   }
  },
  {
   "name": "temperature",
   "type": {
    "type": "float",
    "arg.properties": {
     "range": {
      "min": 0.0,
      "max": 45.0
     }
    }
   }
  },
  {
   "name": "humidity",
   "type": {
    "type": "float",
    "arg.properties": {
     "range": {
      "min": 5.0,
      "max": 100.0
     }
    }
   }
  },
  {
   "name": "soil_fertility",
   "type": {
    "type": "float",
    "arg.properties": {
     "range": {
      "min": 0.0,
      "max": 100.0
     }
    }
   }
  },
  {
   "name": "timestamp",
   "type": {
```

```
        "type": "long",
        "logicalType": "timestamp-millis",
        "arg.properties": {
          "iteration": {
            "start": 1741511477000,
            "step": 1000
          }
        }
      }
    }
  ]
}
```



Los mensajes producidos por el conector anteior, son escritos al topic iot_fields. En este caso no existe data contract, por lo que debemos definir la estructura del mensaje en nuestro fichero de configuración.


iot_fields
Query with Flink
Enable Tableflow
Actions
Messages
Monitor
Data contracts New
Configuration
Production in last hour 6432 messages
Consumption in last hour 70 messages
Total messages 49.051
Retention time 1 week
Partition All
Consume Latest
Max results 1,000
CSV
JSON
Search...
:45
:50
:55
07:12
:05
:10
:15
:20
:25
:30
:35
:40
:45
:50
Timestamp
Partition
Offset
Key
Value
2025-05-22T19:12:51.764Z 1 8207 m}-n□zT{□{... { "key": "m}-n\u001fzT{\u0017{A`", "sensor_id": "#\n!soA[\fo", "temperature": 2.7541513, "humidity": 68.735214, "soil_fer...
2025-05-22T19:12:51.163Z 3 8196 g/dDkg'C□... { "key": "g/dDkg'C\u0003\u001b3mT?j", "sensor_id": "\b9\b2\u0007*\u0003zv", "temperature": 30.46948, "humidity": 35....
2025-05-22T19:12:50.408Z 3 8195 2'E1R]IUu... { "key": "\t2'E1R]IUu\u0002!\u001e\u0005\u000f", "sensor_id": "~\u000e\\Vc\u001eDqR\u0004", "temperature": 24.962...
2025-05-22T19:12:49.829Z 4 8196 r □□RK.'I { "key": "r\t\u001d\u0014RK.'I", "sensor_id": "SW_A{M6`_WIv\u0017", "temperature": 39.64169, "humidity": 86.8987, "soil...
2025-05-22T19:12:49.774Z 5 8143 L8l1P□K- { "key": "L8l1P\bK-", "sensor_id": "\u0014_o;yw\u0001a\u0003\u001dUS;B\u0011", "temperature": 44.295296, "humidity...
2025-05-22T19:12:48.792Z 0 8094 }A□□,C3□3... { "key": "}A\u0013\u0013,C3\u00013r\u0007s\u0007\u0012\u0004", "sensor_id": "B^qq!.)fo\\\u0000?Gu", "temperature"...
2025-05-22T19:12:48.380Z 1 8206 9\□8□□[□ { "key": "9\\\b8\b\u0007{\u0016", "sensor_id": "dG;T?y\u001a\u0006\fl\u0014\u001a\r", "temperature": 32.955685, "hum...
2025-05-22T19:12:48.177Z 3 8194 Z□#DV*b□... { "key": "Z\f#DV*b\u001eo*0d]Gl", "sensor_id": "\u0019Rf\u001a<\u0018?(", "temperature": 40.568283, "humidity": 54.5...
2025-05-22T19:12:47.178Z 5 8142 |^jkp%1i□ { "key": "|^jkp%1i\u0016", "sensor_id": "\u001dn>\u0013\u001d\u0010h4i[¨;D[\u0011", "temperature": 11.308831, "hum...
2025-05-22T19:12:46.808Z 1 8205 <XC□K(my... { "key": "<XC\u0002K(myGm", "sensor_id": "7>'Y}\u001a@Df\u001c", "temperature": 34.620796, "humidity": 78.10767, "s...
2025-05-22T19:12:46.167Z 1 8204 mxs"□Jpa { "key": "mxs\"\u0015Jpa", "sensor_id": "5Q\r\f)YpA)Y", "temperature": 13.214265, "humidity": 62.355156, "soil_fertility": ...
2025-05-22T19:12:46.064Z 4 8195 u□P2Xj@□l { "key": "u\u0014P2Xj@\u001bl", "sensor_id": "\u0013{*\b\u0001lHT\tFa", "temperature": 11.162232, "humidity": 91.466...
2025-05-22T19:12:45.481Z 1 8203 (□X□QMl_... { "key": "(\u0010X\u0018QMl_@/o¨Xh", "sensor_id": "\\0dD;AR\u001b0\bc\r", "temperature": 25.589712, "humidity": 77....
2025-05-22T19:12:45.118Z 2 8209 □□+2RH-□ { "key": "\u0007\u0015+2RH-¨\r", "sensor_id": "n\u000f@\u001coL2,M\u0017\u001a\u0018%G", "temperature": 15.246...
2025-05-22T19:12:45.065Z 0 8093 L□]kU□□D { "key": "L\u0003\t]kU\u001aD", "sensor_id": "-\u0011Wm\"'uO\u000b0Tu\u0016", "temperature": 23.36939, "humidity": ...

Diseño de la arquitectura del Lakehouse

Nos encontramos ante una empresa ficticia en la que el volumen de datos es muy alto, teniendo varias ingestas cada hora, así como varias ingestas streaming. Por ello hemos dedicido optar por una Arquitectura Medalion, estructurada en las capas **Landing, Raw, Bronze, Silver y Gold**, que nos aporta las siguientes ventajas:

1. Separación de responsabilidades y control de calidad

Cada capa cumple un propósito específico en el ciclo de vida del dato, lo que permite establecer **controles de calidad progresivos** y **limitar el impacto de errores** aguas abajo:

- **Landing:** zona de staging. Aquí se almacenan los archivos tal y como llegan, sin ningún tipo de validación ni transformación. Es útil para auditoría, trazabilidad y reingestión.
- **Raw:** copia inalterada pero organizada de los archivos originales. Se estructura de forma más controlada, sin aplicar parseo ni lógicas de negocio.
- **Bronze:** los datos ya son parseados, tipados y almacenados en formato **Delta Lake**, listos para procesamiento analítico. Se filtran duplicados y errores de formato, pero se conserva todo lo relevante.
- **Silver:** los datos se limpian, filtran y normalizan. Aquí se aplican **reglas de negocio**, integraciones con otras fuentes y enriquecimiento semántico.
- **Gold:** datos listos para **consumo analítico** (BI, ML, reporting), con KPIs, agregaciones o modelos dimensionales precomputados.

Este enfoque modular facilita tanto la **escalabilidad horizontal** como la **observabilidad** del pipeline.

2. Trazabilidad y gobernanza de datos

Cada capa guarda una versión explícita del estado del dato, lo que permite:

- Trazar el dato desde su origen hasta su exposición final (lineage).

- Auditar cambios en cada transformación.
- Realizar pruebas A/B o regresión sobre datos históricos.
- Reprocesar o restaurar capas superiores a partir de capas inferiores.

La separación landing/raw es especialmente útil para cumplir con normativas de conservación de datos originales como **GDPR**, **LOPDGDD** o auditorías SOX.

3. Mejora del rendimiento y la eficiencia

Transformar directamente desde datos crudos a métricas es costoso y propenso a fallos. Al utilizar capas intermedias:

- Podemos **almacenar transformaciones intermedias reutilizables** (por ejemplo, joins costosos o limpieza de calidad).
- Los analistas y modelos acceden a las capas Gold/Silver optimizadas para consulta, sin impactar la zona transaccional.
- Se reducen significativamente los **costes de recomputación** y el **time-to-insight**.

4. Soporte nativo para batch y streaming

La arquitectura soporta flujos mixtos de ingestión:

- Datos **batch** (archivos CSV, JSON, Parquet, etc.) que llegan de forma periódica.
- Datos **streaming** (Kafka, Event Hubs, etc.) que se procesan en tiempo real.

Gracias a **Delta Lake**, se garantiza **consistencia transaccional (ACID)**, lo que permite trabajar de forma uniforme con ambos tipos de flujos sobre el mismo sistema de archivos.

5. Escalabilidad y mantenimiento

Cada capa permite:

- Escalar de forma independiente las tareas asociadas a su procesamiento (e.g., parseo, enriquecimiento, agregación).

- Aplicar políticas de **retención diferencial** (por ejemplo, mantener solo 30 días en bronze, 1 año en silver y KPIs indefinidos en gold).
- Modularizar el desarrollo: distintos equipos pueden operar sobre distintas capas con contratos bien definidos.

6. Compatibilidad con arquitectura Lakehouse y herramientas modernas

La arquitectura medallion se alinea con:

- El paradigma **Lakehouse**: combinar lo mejor de los data lakes (flexibilidad, bajo coste) y los data warehouses (estructuración, gobernanza).
- Tecnologías como **Databricks**, **Delta Lake**, **Apache Spark**, **Power BI**, **MLFlow** o **dbt**.
- Integración natural con sistemas de control de versiones y despliegue como **CI/CD** y **infraestructura como código (IaC)**.

Flujo de los datos

Podemos distinguir dos flujos de datos: batch y streaming

- Datos batch

1- Son escritos por un script python que se ejecuta en un notebook a la capa landing.

2- Desde landing hacemos una lectura cada hora de los nuevos archivos que hayan sido ingestados. Estos archivos serán escritos primero a la capa Bronze y una vez escritos en Bronze, serán movidos a Raw. Con ello evitamos que el almacenamiento necesario para landing se incremente. Aplicamos mínimas transformaciones consistentes en añadir metadatos como el nombre de archivo, hora de ingesta y, en caso de ser imágenes, metadatos relativos a ella: alto, ancho, relación de pixeles.

- Datos streaming

Los datos provenientes de Confluent son escritos a la capa landing tal como vienen de Kafka, solo añadiendo la información de a qué hora fueron ingestados a landing en formato parquet. Después de ello, el procesamiento es el mismo, salvo que se parsea para convertir de binario como los recogemos de kafka un formato que podamos persistir

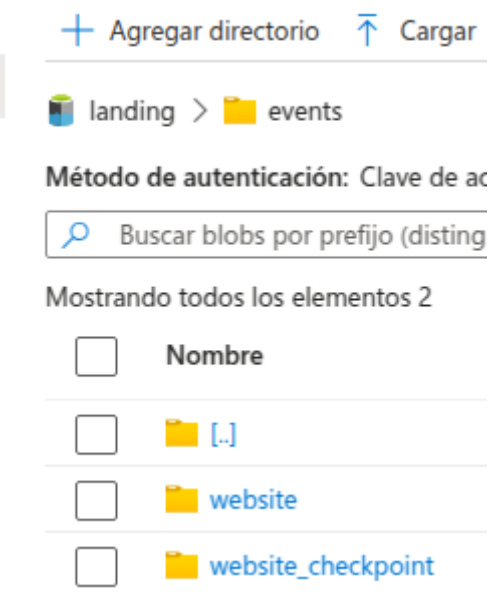
Capas de la arquitectura

Capa landing

En ocasiones esta capa no se considera parte de la arquitectura pero como en este caso, nosotros controlamos la ingestión en ella, la he considerado parte de la arquitectura. En ella la información se organiza por:

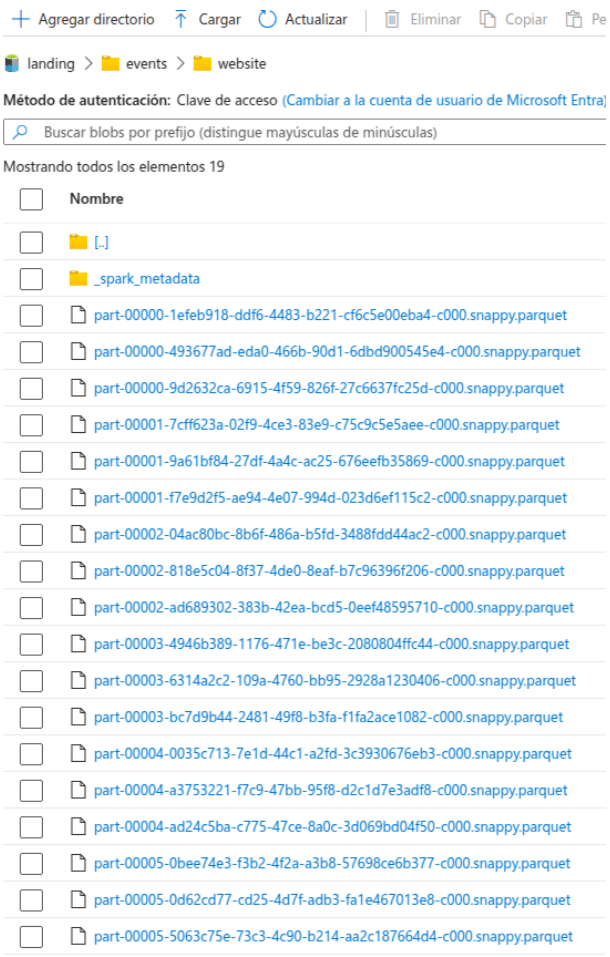
- **Data source:** origen del que provienen los datos (events, iot, weather)
- **Data set:** dato especifico de esa fuente

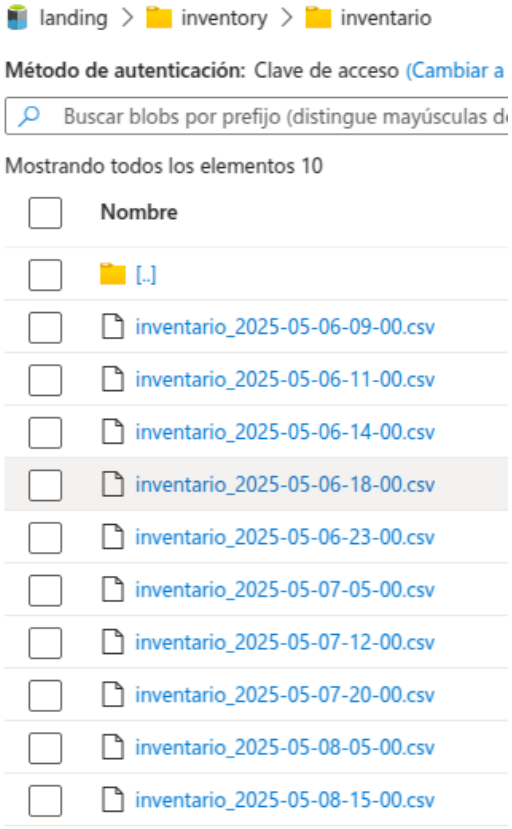
events	=>	website
iot	=>	fields
weather	=>	forecast data



Dentro de la carpeta data_source/dataset, tendremos archivos en el formato especificado anteriormente con la convención de nombres siguiente:

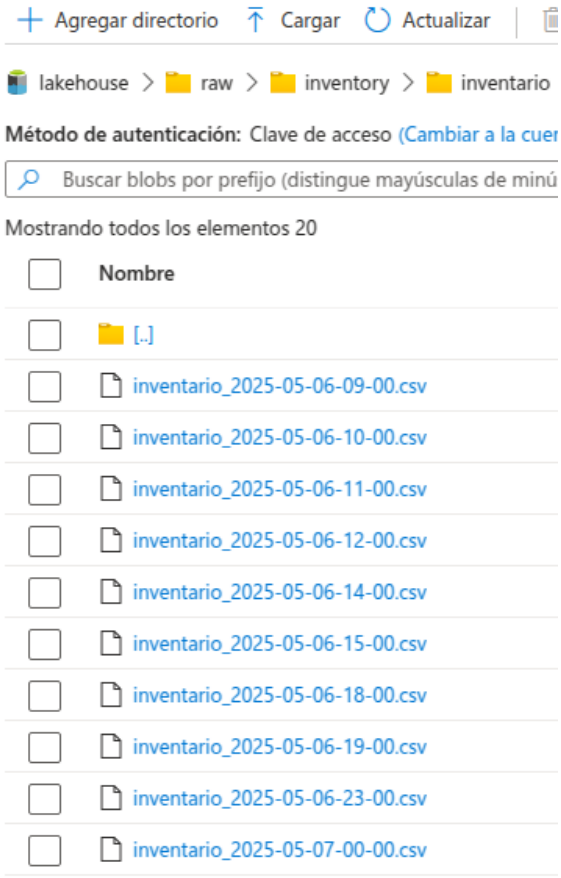
data_source-dataset-year-month-day-hour.csv/json





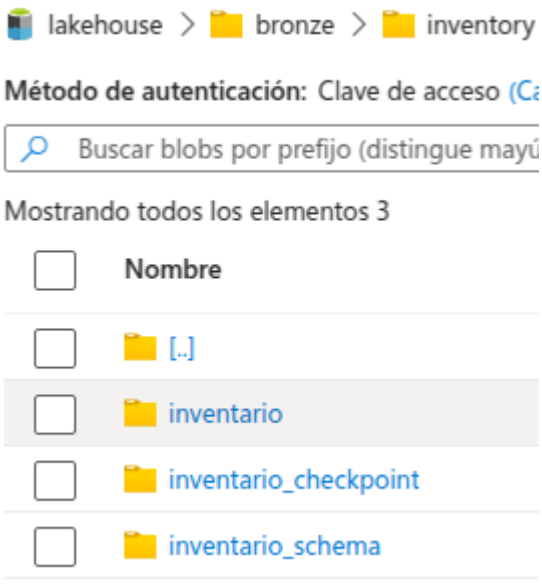
Capa raw:

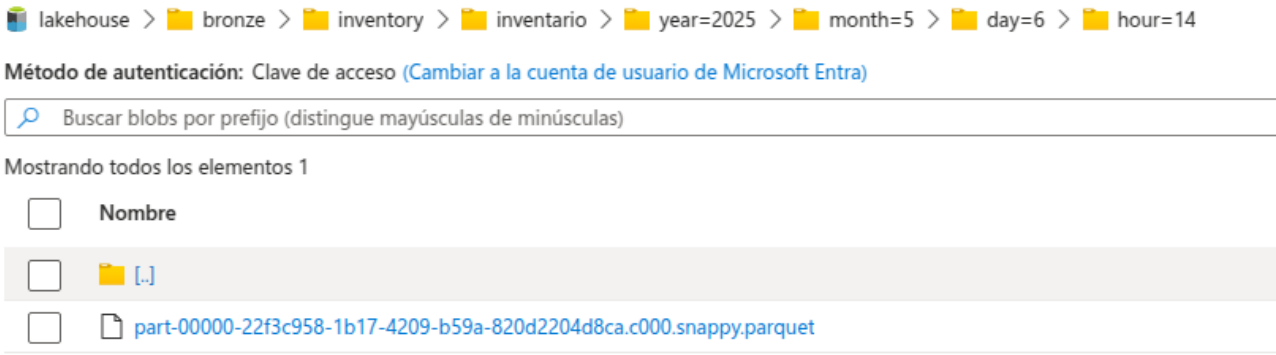
Tendrá la misma estructura interna que la capa landing, ya que al mover los archivos, el path permanece. La extensión de los archivos también será la misma.

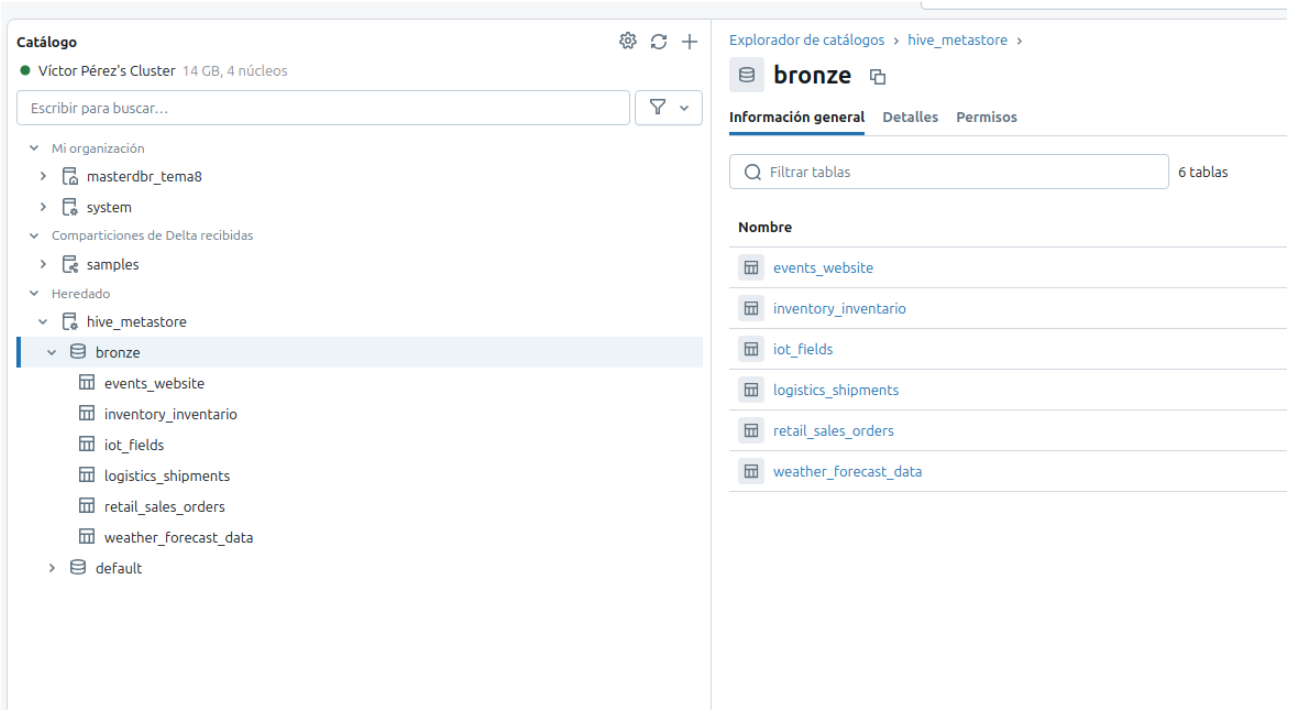


Capa Bronze:

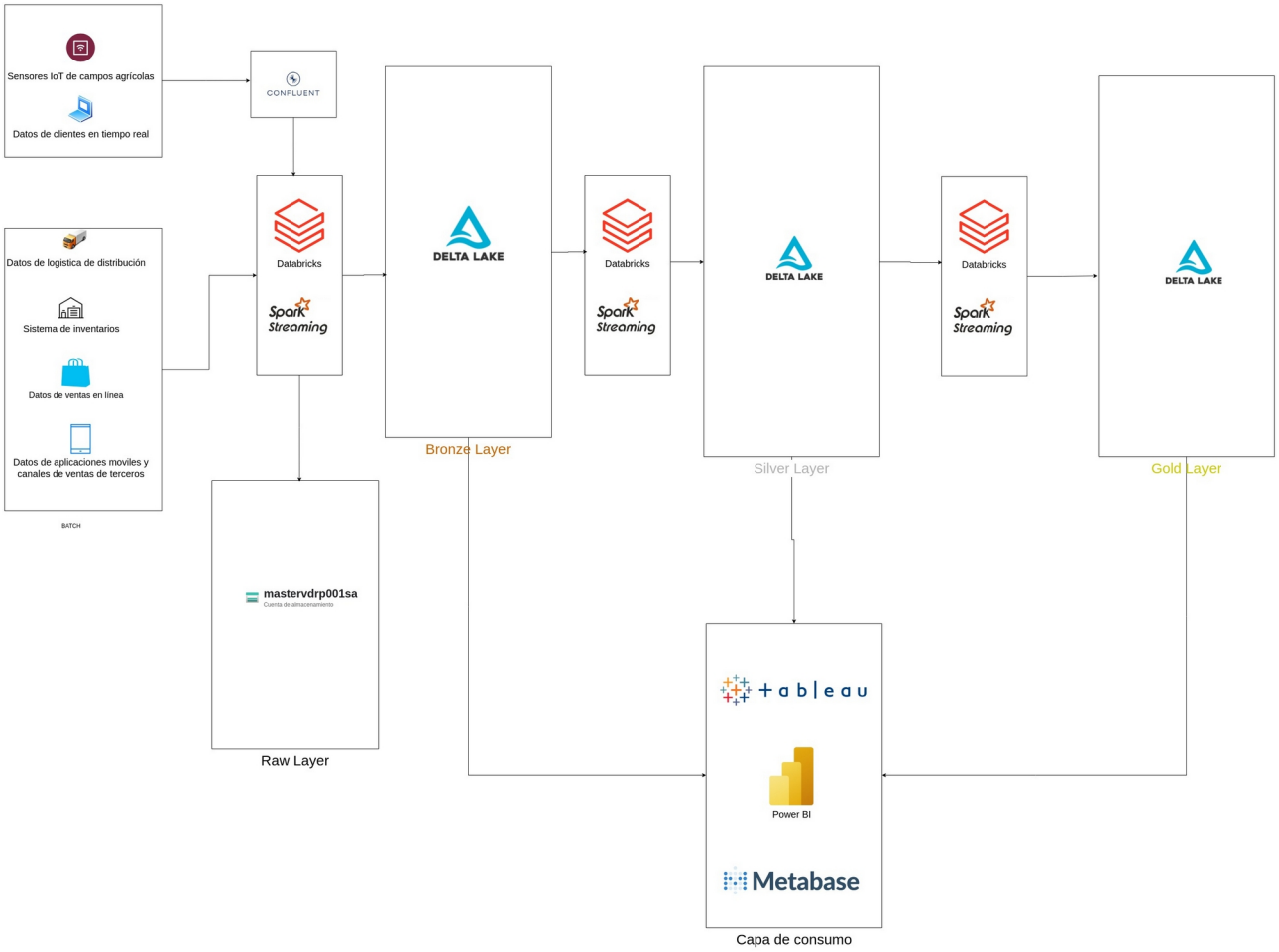
En esta capa ya se almacenan los archivos haciendo uso de formato Delta. Esto nos permite utilizar **Hive Metastore** y que no sea solamente una capa de almacenamiento intermedia, sino que pueda ser consultada usando SQL e incluso herramientas de consumo más usadas habitualmente para capas posteriores (Power BI, Tableau, etc.)




lakehouse > bronze > inventory > inventario > year=2025 > month=5 > day=6 > hour=14
Método de autenticación: Clave de acceso (Cambiar a la cuenta de usuario de Microsoft Entra)
Buscar blobs por prefijo (distingue mayúsculas de minúsculas)
Mostrando todos los elementos 1
Nombre
[..]
part-00000-22f3c958-1b17-4209-b59a-820d2204d8ca.c000.snappy.parquet


Catálogo
Víctor Pérez's Cluster 14 GB, 4 núcleos
Escribir para buscar...
Mi organización
masterdbr_tema8
system
Comparticiones de Delta recibidas
samples
Heredado
hive_metastore
bronze
events_website
inventory_inventario
iot_fields
logistics_shipments
retail_sales_orders
weather_forecast_data
default
Explorador de catálogos > hive_metastore >
bronze
Información general
Detalles
Permisos
Filtrar tablas
6 tablas
Nombre
events_website
inventory_inventario
iot_fields
logistics_shipments
retail_sales_orders
weather_forecast_data

Esquema de la Arquitectura



Implementación del motor de ingesta

La configuración necesaria para la ejecución del código la encontramos en el archivo *azure_config.json,* adjunto en esta entrega. La descripción del mismo es la siguiente:

Nivel 1: Parámetros Globales

Estos parámetros aplican a toda la configuración del proyecto:

- DATABRICKS_CONFIG_PROFILE: define el perfil de Databricks a utilizar (DATAENGINEERING).
- allowed_file_formats: lista formatos de archivo permitidos globalmente (json, csv, avro, parquet, orc, binary, text).
- token_cuenta_almacenamiento: Token cifrado (en este caso ficticio) para autenticación en Azure Blob Storage.
- schema_registry_api_credentials: Credenciales para conectarse a Confluent Schema Registry, obligatorio para leer datos en formato Avro desde Kafka.

Nivel 2: data_type_information

Aquí defines de forma específica cómo manejar cada uno de los datasets organizados en diferentes **dominios**:

Dominios disponibles:

- retail
- inventory
- logistics
- weather
- events
- iot

Cada dominio contiene uno o varios datasets con esta estructura:

- **Rutas de almacenamiento**:
    - landing_path: Ubicación inicial (landing zone).
    - raw_path: Ubicación histórica (raw zone).
    - bronze_path: Ubicación post-procesamiento inicial (bronze zone).
- **Especificaciones**:
    - processing_type: Puede ser "batch" o "streaming".

- dataset_format: Formato de archivo (json, parquet, etc.).
- partition_columns: Columnas para particionar en formato Delta (variable según dataset, temporal o no).
- schema_json: Esquema detallado del dataset.

Ejemplos:

Retail – Dataset: sales_orders

- **Batch**
- Formato: JSON
- Ruta Landing: abfss://landing@.../retail/sales_orders
- Esquema definido (ej.: order_id, customer_id, unit_price, etc.).

Inventory – Dataset: products

- **Batch**
- Formato: CSV
- Usa esquema explícito con columnas como producto_id, nombre_producto, etc.

Events – Dataset: website

- **Streaming desde Kafka**
- Formato Avro (con Schema Registry)
- Especifica Topic Kafka (events-website)
- Configuración adicional específica para Kafka (key_format, value_format).

IoT – Dataset: fields

- **Streaming desde Kafka**
- Formato JSON (sin Schema Registry)
- Especifica Topic Kafka (iot-fields)
- Esquema detallado para datos de sensores agrícolas

Para hacer uso de esta configuración, utilizamos un método de clase, el cual organiza toda la configuración, comprueba su validez y procede a crear el objeto de la clase necesaria. Tenemos este por ejemplo:

```python
@classmethod
def from_config(
    cls,
    datasource_config_path: str,
    kafka_config_path: str,
    data_source: str,
    dataset: str
) -> "KafkaStreamReader":
    """
    Crea una instancia a partir de varios archivos de configuración.
    Args:
        datasource_config_path: Ruta al config de datasource.
        kafka_config_path: Ruta al client.properties de Kafka.
        schema_registry_conf_path: Ruta al config de schema registry.
        data_source: Nombre de la fuente de datos.
        dataset: Nombre del dataset.
    Returns:
        Instancia de KafkaStreamReader.
    """
    try:
        logger.info(f"Creando instancia de KafkaStreamReader para {data_source}/{dataset}")

        datasource_config = cls.get_datasource_config(datasource_config_path)
        kafka_config = cls.get_kafka_config(kafka_config_path)

        # comprobamos que la configuración venga correctamente antes de empezar
        cls.validate_kafka_config(datasource_config, data_source, dataset, kafka_config)

        return cls(
            kafka_config=kafka_config,
            data_source=data_source,
            dataset=dataset,
            bronze_path=datasource_config['data_type_information'][data_source][dataset]['bronze_path'],
            key_format=datasource_config['data_type_information'][data_source][dataset]['key_format'],
            value_format=datasource_config['data_type_information'][data_source][dataset]['value_format'],
            topic=datasource_config['data_type_information'][data_source][dataset]['topic'],
            schema_registry_conf=(
                datasource_config['schema_registry_api_credentials'])
        )
    except Exception as e:
        logger.exception(f"Error en KafkaStreamReader.from_config: {e}")
        raise
```

Integración de Datos Batch

A la hora de implementar el código, se ha decidido utilizar jobs en Azure Databricks. Debido a la falta de tiempo, no se ha podido generar un paquete .whl funcional para instalar en el cluster de Databricks, por lo que se ha optado por lanzar jobs que ejecuten notebooks de Databricks, facilitando en cierta manera la implementación.
La configuración necesaria para la ejecución del código la encontramos en el archivo *azure_config.json* mencionado anteriormente y adjunto a la tarea. Hemos conseguido un motor de ingesta modular y bien definido, que nos permite:

- ingestar archivos de varios formatos:

```
if self.format == "csv":
   return (spark.readStream
           .format("cloudFiles")
           .option("cloudFiles.format", self.format)
           .option("header", "true")
           .schema(self.schema_json)
           .option("recursiveFileLookup", "true")
           .option("cloudFiles.schemaLocation", self.bronze_schema_location)
           .option("rescuedDataColumn", "_rescued_data")
           .load(self.landing_path)
           )
elif self.format == "binary":
   return (
      spark.readStream
      .format("cloudFiles")
      .option("cloudFiles.format", "binaryFile")
      .option("pathGlobFilter", "*.jpg")
      .option("recursiveFileLookup", "true")
      .option("cloudFiles.schemaLocation", self.bronze_schema_location)
      .load(self.landing_path)
   )
elif self.format in {"json", "avro", "parquet"}:
   return (spark.readStream
           .format("cloudFiles")
           .option("cloudFiles.format", self.format)
           .schema(self.schema_json)
           .option("cloudFiles.schemaLocation", self.bronze_schema_location)
           .option("recursiveFileLookup", "true")
           .load(self.landing_path)
```

```
        )
else:
    raise ValueError(f"Formato de archivo '{self.format}' no soportado.")
```

- Definir el esquema esperado en el archivo de configuración. Por ejemplo:

```
"schema_json": {
  "fields": [
    {
      "name": "fecha",
      "type": "string",
      "nullable": false,
      "metadata": {
        "description": "Fecha de la predicción meteorológica",
        "format": "yyyy-MM-dd'T'HH:mm:ss"
      }
    },
    {
      "name": "humedad",
      "type": "integer",
      "nullable": false,
      "metadata": {
        "description": "Porcentaje de humedad relativa",
        "units": "%"
      }
    },
    {
      "name": "precipitacion_mm",
      "type": "double",
      "nullable": false,
      "metadata": {
        "description": "Cantidad de precipitación en milímetros",
        "units": "mm"
      }
    },
    {
      "name": "region",
      "type": "string",
      "nullable": false,
      "metadata": {
        "description": "Zona geográfica"
      }
    },
    {
      "name": "temperatura",
      "type": "double",
      "nullable": false,
      "metadata": {
        "description": "Temperatura en grados Celsius",
        "units": "°C"
```

```
    }
   },
   {
    "name": "velocidad_viento_kmh",
    "type": "double",
    "nullable": false,
    "metadata": {
     "description": "Velocidad del viento en km/h",
     "units": "km/h"
    }
   },
   {
    "name": "timestamp",
    "type": "timestamp",
    "nullable": false,
    "metadata": {
     "description": "Marca temporal del evento"
    }
   }
  ]
}
```

- Configurar rutas de origen (landing), de almacenamiento raw y bronze, así como las columnas de particionado, el tipo de procesamiento y el formato del datase:

```
"processing_type": "batch",
"landing_path": "abfss://landing@mastervdrp001sa.dfs.core.windows.net/weather/forecast_data",
"raw_path": "abfss://lakehouse@mastervdrp001sa.dfs.core.windows.net/raw/weather/forecast_data",
"bronze_path": "abfss://lakehouse@mastervdrp001sa.dfs.core.windows.net/bronze/weather/
forecast_data",
"dataset_format": "json",
"partition_columns": ["year", "month", "day", "hour"],
```

- Agregar metadatos a cada registro:

```
def add_metadata_columns(self, df: DataFrame) -> DataFrame:
    """
    Añade columnas de metadatos y de particionado temporal al DataFrame.
    Args:
        df: DataFrame de entrada.
    Returns:
        DataFrame con columnas adicionales de metadatos.
    Raises:
        ValueError: Si no existe la columna 'timestamp' en el DataFrame.
    """
    try:
        if self.format == "binary":
            df = self.add_image_metadata(df)
```

```
        data_cols = df.columns
        if "timestamp" not in data_cols:
            logger.error("Falta columna 'timestamp' en el DataFrame.")
            raise ValueError(
                "Error: el DataFrame no contiene la columna timestamp. "
                "Revisa el esquema de entrada.")

        metadata_cols = ['_ingested_at', '_ingested_filename', 'year', 'month', 'day', 'hour']

        logger.info("Añadiendo columnas de metadatos al DataFrame.")
        df = (df.withColumn("_ingested_at", F.current_timestamp())
              .withColumn("_ingested_filename", F.input_file_name())
              .withColumn("year", F.year("timestamp"))
              .withColumn("month", F.month("timestamp"))
              .withColumn("day", F.dayofmonth("timestamp"))
              .withColumn("hour", F.hour("timestamp"))
              )
        return df.select(metadata_cols + data_cols)
    except Exception as e:
        logger.exception(f"Error en LandingBatchReader.add_metadata_columns: {e}")
        logger.exception(f"Error en LandingBatchReader.add_metadata_columns: {e}")
        raise
```

Permitir evolución de esquema si los cambios son compatibles y usar Databricks Autoloader:

```
(df.write
    .format("delta")
    .option("mergeSchema", "true")
    .option("path", self.bronze_path)
    .partitionBy(*self.partition_columns)
    .mode("append")
    .save()
)
```

Registrar los datos en Hive Metastore:

```
spark.sql('CREATE DATABASE IF NOT EXISTS hive_metastore.bronze')
spark.sql(f"CREATE TABLE IF NOT EXISTS {self.table} USING DELTA LOCATION '{self.bronze_path}'")
```

Integración de Datos Streaming

Los mensajes que nos llegan desde Kafka, son ingestados si deserializar en nuestro almacenamiento Landing en formato parquet, para luego ser leídos de ahí, deserializados y almacenados en bronze. Antes de ser deserializados, son movidos al almacenamiento RAW.

Durante el proceso de deserialización, se añaden metadatos y se extraen de la columna '*_value'* para que quede una estructura plana.

En el archivo de configuración se establece a su vez cada cuanto se dispara (trigger) el proceso. En caso de no aparecer esta configuración, ejecutaremos el proceso continuamente. Esta última opción no es aconsejable ya que con ella incurriremos en gastos extra. Podemos asignar un trigger para que se ejecute desde 1 segundo, hasta el tiempo que queramos, dándonos así flexibilidad en función de la "urgencia" del tipo de dato.

Permite ingestar datos en avro y json

```
if self.value_format == "avro":
    value_subject = f"{self.topic}-value"
    schema_str = SchemaRegistryClient(self.schema_registry_conf).get_latest_version(
        value_subject).schema.schema_str
    df = (df
          .withColumn("value", from_avro(F.expr("substring(_value,6,length(_value)-5)"), schema_str))
         )

elif self.value_format == "json":
    if not self.schema_json:
        raise ValueError("Falta 'value_schema' para deserializar JSON plano desde Kafka.")
    df = (
        df  .withColumn("_timestamp", F.col("_timestamp").cast("timestamp"))
            .withColumn(
                "value",
                F.from_json(
                    F.col("_value").cast("string"),
                    StructType.fromJson(self.schema_json))
            )
        )
else:
    # si alcanzamos esta línea, algo está fallando al parsear
    raise ValueError(
        f"No se ha podido parasear el DataFram."
        f"DatasetFormat '{self.value_format}' no permitido."
        f"Datasource: '{self.data_source} Dataset:{self.dataset}'."
        f"Formatos permitidos: {self.allowed_file_formats}."
    )
```

```
return (df.drop("_value")
          .select("*", "value.*")
          .drop("value")
       )
```

Key y value subject. Para parsear la key, espesificamos el formato en que nos llega en al configuración, para ser utiliado en este codigo:

```
def parse_key(self, df: DataFrame) -> DataFrame:
    """
    Parsea el campo `key` del DataFrame según el formato configurado.
    Args:
        df (DataFrame): DataFrame leído desde Kafka, con columna `_key` en formato binario.
    Returns:
        DataFrame: DataFrame con la columna `key` ya parseada y tipada.
    """
    try:
        logger.info("Deserializando campo 'key'")

        if self.key_format == "int":
            return df.withColumn("_key", F.col("_key").cast("int"))

        elif self.key_format == "string":
            return df.withColumn("_key", F.col("_key").cast("string"))

        elif self.key_format == "json":
            if not self.schema_json:
                raise ValueError("Se esperaba un schema JSON para la clave, pero no se encontró
'key_schema'.")
            return df.withColumn(
                "_key",
                F.from_json(F.col("_key").cast("string"), StructType.fromJson(self.schema_json))
            )

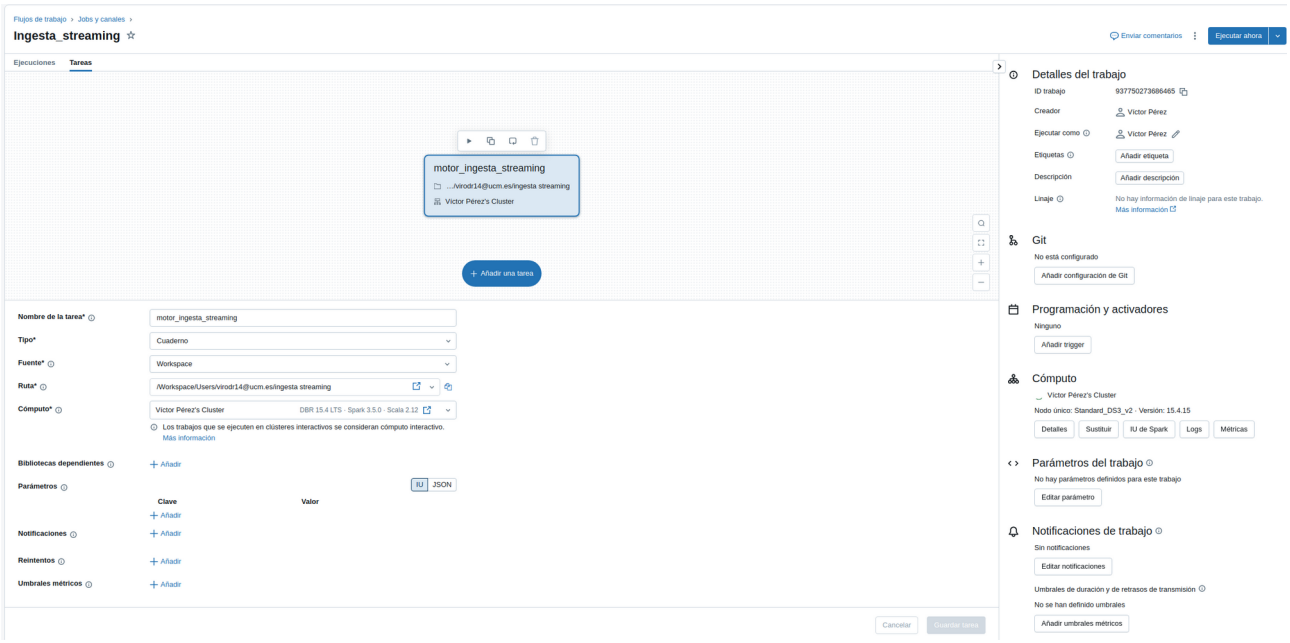
        else:
            logger.warning(f"Formato de clave '{self.key_format}' no reconocido. Se deja la columna `_key`
sin parsear.")
            return df
```

Deserialización de datos de Kafka

Nuestro código permite dos tipos de serialización:

- avro: establecemos un data contract en el Schema Registry de Confluent y lo consultamos a la hora deserializar. Permite evolución Backwards
- json: NO establecemos Data Contract, definimos la estructura de los mensajes json en la configuración.

Ejemplo de configuración de jobs



Configuración requerida:

- azure_config.json: el grueso de la configuración. En este caso incluye tokes de acceso tanto a las cuentas de almacenamiento de Azure, schema_registry_api_credentials

- client.properties: para poder conectarnos con Confluent

- Data source/dataset: definidos como parametros del job, son necesarios para “navegar” por la configuración.

Mejoras pendientes

- Generar archivo .whl para instalar en el cluster y lanzar scripts de Python desde los jobs de Azure Databricks.

- Mejor redacción y finalización tanto de los notebooks, como de la memoria de la tarea que ha tenido la paciencia de leer.